AMPLIACIÓN · NO EVALUABLE

UD02 — Ampliación: primeros pasos con Docker

Módulo: Sistemas Operativos Monopuesto · 1º SMR

Actividad de ampliación, no evaluable (RA5). No cuenta para la nota. Sirve para conocer otra forma de virtualizar que te vas a encontrar en las empresas: los **contenedores**. Está pensada para hacerla **por tu cuenta** en las sesiones del 11 al 18 de diciembre, siguiendo los pasos en orden. El lunes 21 la pondremos en común en clase.

Si tienes alguna práctica de la unidad pendiente o suspendida, dedica estas sesiones a terminarla o rehacerla: tiene prioridad sobre esta ampliación.

Qué vas a aprender

En el epígrafe 6 de los apuntes viste la diferencia entre una máquina virtual y un **contenedor**:

- Una **máquina virtual** simula un ordenador completo y lleva dentro su propio sistema operativo entero, con su propio núcleo. Por eso ocupa varios gigabytes y tarda un minuto o más en arrancar.
- Un **contenedor** no simula ningún ordenador. Es un conjunto de procesos aislados del resto del sistema, que **comparten el núcleo** del sistema en el que se ejecutan. Solo empaqueta la aplicación y lo que esta necesita (bibliotecas, configuración). Por eso ocupa unos megas y arranca en un segundo.

Docker es la herramienta más usada para crear y ejecutar contenedores. Para entenderlo, necesitas dos palabras:

- **Imagen:** la "plantilla" de solo lectura con la aplicación y todo lo que necesita. Por ejemplo, la imagen `nginx` contiene un servidor web listo para funcionar.
- **Contenedor:** una **copia en marcha** de una imagen. De una misma imagen puedes arrancar tantos contenedores como quieras, igual que de una misma ISO puedes instalar muchas VMs.

En esta ficha vas a instalar Docker **dentro de tu VM de Ubuntu**. Así no tocas el equipo del aula y, además, vas a ver algo interesante: un contenedor funcionando dentro de una máquina virtual, una capa de virtualización encima de otra.

Antes de empezar

1. **Toma una instantánea** de tu VM `Apellido-UD02-Linux` con el nombre "Antes de Docker". Si algo sale mal, vuelves a ella y empiezas de nuevo.
2. Comprueba que la VM tiene **al menos 2 GB de RAM** y **salida a internet** (modo de red NAT).

3. Ten a mano el archivo **`imagenes_som.tar`** que te ha dejado tu profesor en Moodle o en la carpeta compartida. Contiene las imágenes que vas a usar, para no tener que descargarlas de internet (ver el paso 3).

En todos los pasos, los comandos se escriben en una **terminal de Ubuntu** (`Ctrl + Alt + T`).

Paso 1. Instalar Docker

Ubuntu incluye Docker en sus repositorios oficiales, con el nombre de paquete `docker.io`. Se instala con APT, igual que cualquier otro programa:

```
sudo apt update
sudo apt install docker.io
```

Docker funciona como un **servicio**: un programa que se queda en marcha en segundo plano (el *demonio* de Docker), esperando órdenes. Comprueba que está funcionando:

```
systemctl status docker
docker --version
```

✅ **Comprobación:** `systemctl status docker` debe mostrar **`active (running)`** en verde (sal con la tecla `q`), y `docker --version` debe mostrar la versión instalada.

Paso 2. Usar Docker sin `sudo`

Para hablar con el servicio de Docker hacen falta permisos de administrador. Para no escribir `sudo` delante de cada orden, añade tu usuario al grupo `docker`:

```
sudo usermod -aG docker $USER
```

El cambio de grupo no se aplica hasta que vuelves a iniciar sesión. **Cierra la sesión de Ubuntu y vuelve a entrar** (o reinicia la VM). Después, comprueba que perteneces al grupo:

```
groups
```

✅ **Comprobación:** en la lista que muestra `groups` aparece **`docker`**.

> ⚠️ **Ojo con la seguridad.** Pertenecer al grupo `docker` equivale, en la práctica, a ser administrador del sistema, porque con Docker se puede acceder a cualquier archivo del equipo. En tu VM de prácticas no hay problema. En un servidor de verdad, solo se mete en este grupo a quien se le daría también la contraseña de administrador.

Paso 3. Cargar las imágenes

Las imágenes de Docker se descargan normalmente de **Docker Hub**, un repositorio público en internet. Pero Docker Hub limita cuántas descargas se pueden hacer desde una misma dirección IP, y en el instituto todos salimos a internet por la misma. Por eso tu profesor ha descargado las imágenes antes y las ha guardado en un solo archivo.

Copia `imagenes_som.tar` a tu carpeta personal de la VM (desde la carpeta compartida de la Práctica 5 o descargándolo de Moodle) y cárgalo:

```
docker load -i ~/imagenes_som.tar
docker images
```

✅ **Comprobación:** `docker images` muestra tres imágenes: **`hello-world`**, **`alpine`** y **`nginx`** (con la etiqueta `alpine`), con su tamaño. Fíjate en lo poco que ocupan comparadas con tus VMs.

Paso 4. Tu primer contenedor

```
docker run hello-world
```

`docker run` hace dos cosas: **crea** un contenedor a partir de la imagen y lo **arranca**. Este contenedor solo escribe un mensaje y termina.

✅ **Comprobación:** aparece el mensaje **`Hello from Docker!`**. Léelo entero: explica, paso a paso, lo que acaba de hacer Docker.

Ahora lista los contenedores:

```
docker ps
docker ps -a
```

`docker ps` muestra solo los contenedores **en marcha** (no aparece ninguno); `docker ps -a` muestra **todos**, también los terminados. El contenedor de `hello-world` aparece con el estado `Exited`: ya terminó, pero sigue existiendo.

Paso 5. Entrar en un contenedor

Alpine es una distribución de Linux minúscula, de unos pocos megas, muy usada como base de contenedores. Arranca un contenedor de Alpine y entra en su terminal:

```
docker run -it --name prueba alpine sh
```

- `-it` conecta tu teclado y tu pantalla con el contenedor, para poder escribir dentro.
- `--name prueba` le pone un nombre, para no tener que usar su identificador.

- `sh` es el programa que se ejecuta dentro: una terminal.

El indicador cambia a algo como `/ #`: **estás dentro del contenedor.** Ejecuta, uno a uno:

```
cat /etc/os-release
uname -r
ps
ls /
exit
```

✅ **Comprobación:**

- `cat /etc/os-release` dice **Alpine Linux**, aunque estés en una VM de Ubuntu.
- `uname -r` muestra la versión del **núcleo**. Apúntala.
- `ps` muestra solo uno o dos procesos: dentro del contenedor no se ve nada más del sistema.

Ya fuera del contenedor, de vuelta en Ubuntu, ejecuta también `uname -r` y compara con lo que apuntaste. **Este es el paso más importante de toda la ficha**: piensa por qué sale lo que sale (pregunta 1 de la puesta en común).

Para volver a entrar en el mismo contenedor y después borrarlo:

```
docker start -ai prueba
exit
docker rm prueba
```

Paso 6. Un servidor web en un contenedor

Ahora vas a arrancar **nginx**, un servidor web muy usado en internet, y a verlo desde el navegador de tu VM:

```
docker run -d --name web -p 8080:80 nginx:alpine
docker ps
```

- `-d` (*detached*) lo deja funcionando en segundo plano, sin ocupar la terminal.
- `-p 8080:80` **publica un puerto**: lo que llegue al puerto 8080 de tu VM se envía al puerto 80 del contenedor, que es donde escucha nginx. Es la misma idea que la redirección de puertos que puede hacer VirtualBox con el modo NAT.

Abre **Firefox** dentro de la VM y escribe en la barra de direcciones `http://localhost:8080`.

✅ **Comprobación:** aparece la página **"Welcome to nginx!"**, y `docker ps` muestra el contenedor `web` con el estado `Up`.

Prueba también estos comandos y fíjate en lo que hace cada uno:

```
docker logs web
docker stop web
```

```
docker ps -a
docker start web
```

`docker logs` muestra el registro del servidor: verás una línea por cada vez que cargaste la página. Con el contenedor parado, la página deja de cargar; al arrancarlo, vuelve.

Paso 7. Tu propia página web

Vas a hacer que nginx sirva una página escrita por ti. Primero, créala en tu carpeta personal de Ubuntu:

```
mkdir ~/miweb
echo "<h1>Hola, soy $(whoami) y esta web sale de un contenedor</h1>" >
~/miweb/index.html
```

Ahora arranca otro contenedor de nginx que use esa carpeta:

```
docker run -d --name miweb -p 8081:80 -v ~/miweb:/usr/share/nginx/html:ro
nginx:alpine
```

`-v ~/miweb:/usr/share/nginx/html:ro` **monta** tu carpeta `~/miweb` dentro del contenedor, en la ruta donde nginx busca las páginas. `:ro` (*read only*) hace que el contenedor solo pueda leerla, no modificarla. Es la misma idea de "montar" que verás en UD03 con los discos.

Abre `http://localhost:8081` en Firefox.

✅ **Comprobación:** aparece tu frase con tu nombre de usuario.

Ahora modifica la página **sin tocar el contenedor**:

```
echo "<p>Página modificada a las $(date +%H:%M)</p>" >> ~/miweb/index.html
```

Recarga la página en Firefox (`F5`). El cambio aparece al momento, porque el contenedor lee el archivo directamente de tu carpeta.

Paso 8. Limpiar

Los contenedores parados siguen ocupando espacio. Aprende a recoger:

```
docker ps -a
docker stop web miweb
docker rm web miweb
docker ps -a
docker system df
```

`docker system df` muestra cuánto espacio ocupan las imágenes y los contenedores. Las imágenes **no** se borran al borrar los contenedores. Déjalas, porque las vas a usar en el reto. Si algún día quieres borrar una imagen, se hace con `docker rmi nombre-de-la-imagen`.

✅ **Comprobación:** `docker ps -a` ya no muestra los contenedores `web` ni `miweb`, y tu carpeta `~/miweb` sigue intacta.

Reto (opcional): crea tu propia imagen

Hasta ahora has usado imágenes hechas por otros. Una imagen propia se describe en un archivo de texto llamado **`Dockerfile`**: una "receta" que dice de qué imagen partir y qué añadirle.

1. En la carpeta `~/miweb`, crea un archivo llamado `Dockerfile` (sin extensión), por ejemplo con `nano ~/miweb/Dockerfile`, con estas dos líneas:

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
```

`FROM` indica la imagen de partida y `COPY` copia tu página **dentro** de la nueva imagen.

2. Construye la imagen, dándole un nombre y una versión, y arráncala:

```
cd ~/miweb
docker build -t miweb:1.0 .
docker images
docker run -d --name mia -p 8082:80 miweb:1.0
```

El punto final de `docker build` es la carpeta donde está el `Dockerfile` (la actual).

3. Abre `http://localhost:8082`. Después, modifica `index.html` y recarga la página: **esta vez no cambia**. ¿Por qué? (Pregunta 5.)

✅ **Comprobación:** `docker images` muestra tu imagen `miweb` con la etiqueta `1.0`, y en el puerto 8082 se ve tu página.

Si algo falla

Síntoma	Causa probable	Solución
`permission denied ... /var/run/docker.sock`	Tu usuario todavía no está en el grupo `docker` (no has vuelto a iniciar sesión tras el paso 2)	Cierra sesión y vuelve a entrar. Comprueba con `groups`. Mientras tanto, puedes poner `sudo` delante
`Cannot connect to the Docker daemon`	El servicio de Docker no está en marcha	`sudo systemctl start docker` y comprueba con `systemctl status docker`
`Unable to find image ... locally` y después un error de descarga o `toomanyrequests`	La imagen no está cargada y Docker intenta descargarla de internet	Repite el paso 3 (`docker load`). Comprueba con `docker images`
`docker load` dice que no encuentra el archivo	La ruta del `.tar` no es correcta	Comprueba con `ls ~` dónde está el archivo y escribe su ruta exacta

Síntoma	Causa probable	Solución
`port is already allocated`	Otro contenedor ya usa ese puerto de la VM	Usa otro puerto (por ejemplo `-p 8090:80`) o para el otro contenedor con `docker stop`
`The container name "/web" is already in use`	Ya existe un contenedor con ese nombre, aunque esté parado	`docker rm web` (o `docker rm -f web` si está en marcha) y vuelve a crearlo
El contenedor de Alpine termina nada más arrancar	Falta `-it`: sin terminal conectada, `sh` no tiene nada que hacer y termina	Arráncalo con `docker run -it ... alpine sh`
`localhost:8080` no carga	El contenedor está parado o el puerto no es el que crees	`docker ps`: el contenedor debe estar `Up` y en la columna `PORTS` debe aparecer `0.0.0.0:8080->80/tcp`
En el paso 7 la página muestra "Welcome to nginx!" y no la tuya	La carpeta montada no es la correcta o `index.html` no está dentro	`ls ~/miweb` y revisa la ruta de `-v`

Si después de esto sigue sin funcionar, apunta el síntoma, el comando que escribiste y el mensaje exacto de error: lo resolveremos el lunes 21. Si la VM queda en mal estado, vuelve a la instantánea "Antes de Docker".

Para la puesta en común del lunes 21

Trae pensadas, o mejor apuntadas, las respuestas a estas preguntas. No se entregan ni se califican.

1. En el paso 5, ¿qué núcleo mostraba `uname -r` dentro del contenedor de Alpine y cuál en tu Ubuntu? ¿Por qué? ¿Qué te dice eso sobre la diferencia entre un contenedor y una máquina virtual?
2. En la Práctica 6 mediste cuánto tarda en arrancar tu VM. Ahora ejecuta `time docker run --rm alpine echo hola`, que mide cuánto tarda en crearse, arrancar, ejecutar una orden y terminar un contenedor. Compara las dos cifras y explica la diferencia.
3. Con lo que has respondido en la pregunta 1, ¿podrías ejecutar en tu Ubuntu un contenedor de **Windows**? ¿Por qué?
4. En el paso 7, si borras el contenedor `miweb`, ¿se pierde tu página? ¿Y si hubieras escrito la página **dentro** del contenedor, sin montar ninguna carpeta?
5. En el reto, ¿por qué al modificar `index.html` la página del puerto 8082 no cambia, y la del 8081 del paso 7 sí?
6. Una empresa quiere que cada desarrollador pruebe su web en el mismo entorno exacto que el servidor. ¿Le recomendarías una VM o un contenedor? ¿Y si lo que necesita es probar una actualización de Windows 11 antes de instalarla en todos sus equipos?